

Resources

Porter Stemmer

The Porter Stemmer is a classic algorithm in Natural Language Processing (NLP) that reduces English words to their root or base form by systematically removing common suffixes.

- The Porter Stemmer is an algorithm developed by Martin Porter in 1980 for stemming English words—that is, reducing various forms of a word (like "running," "runner," "ran") to their common base ("run") for text processing tasks.
- It is widely used in information retrieval and text mining, helping to match similar terms and reduce vocabulary size
- The algorithm operates through a sequence of rule-based steps (usually five major steps), each designed to remove specific types of suffixes, such as "-ing," "-ed," "-ly," etc.
- At each step, the algorithm applies conditions (like presence of vowels/consonants and word length) before removing suffixes to ensure accuracy
- For example:
 - "running" → "run"
 - "happily" → "happi"
 - "characterization" → "character".
- Used in search engines and document retrieval systems to improve matching and searching of word variants
- Prepares data for further processing in NLP pipelines by standardizing word representations.
- Simple to implement
- Handles only English and is not language-agnostic.
- Fast and efficient for large datasets.

Limitations

- Sometimes produces stems that are not actual words (e.g., "happily" → "happi," "university" → "univers"), which may reduce interpretability
- Not always context-aware—different words may be reduced to the same root even if they are not related.
- Can be too aggressive in some cases, merging unrelated word forms

The Porter Stemmer algorithm consists of five key sequential steps, each focusing on reducing different types of suffixes from English words.

Five Main Steps of Porter Stemmer

Step 1: Plural and Past Tense Removal

- Substeps a, b, and c handle common morphological variants:
 - a) Removes plurals ("cats" → "cat") and participles ("running" → "run").
 - b) Converts 'ies'/'ied' endings to 'y'/'i', and strips 'ing', 'ed', etc., if certain conditions are met (such as a vowel-consonant sequence).
 - c) Converts terminal "y" to "i" in some cases.

Step 2: Suffix Substitution

- Replaces a set of longer suffixes if the base form meets a 'measure' condition (number of vowel-consonant sequences $m > 0$):
 - "ization" → "ize", "ational" → "ate", "fulness" → "ful", "ousness" → "ous", etc.

Step 3: Further Suffix Reduction

- Removes or replaces adverbial and adjectival suffixes if the word has sufficient length:
 - "icate" → "ic", "ative" → (deleted), "alize" → "al", "fulness" → "", "ness" → "".

Step 4: More Suffix Removal

- Removes further endings if the stem is long enough ($m > 1$):
 - "ement" → "", "ance" → "", "ment" → "", "ion" (only if preceded by 's' or 't') → "[removed]", etc.

Step 5: Final Adjustment

- Cleans up residual endings in two subparts:
 - Removes final "e" if criteria are met, handles double consonants at the end ("happi" → "happy") and ensures stem stability

Lemmatizer

- Lemmatization is a key natural language processing (NLP) technique that reduces words to their base or dictionary form, called a lemma, based on the word's meaning and context.
- Unlike stemming, which chops off word endings blindly, lemmatization considers the part of speech and the word's intended meaning to return valid root words.
- Lemmatization normalizes different forms of a word to a common base form (lemma). For example, “running,” “ran,” and “runs” are reduced to “run,” and “better” becomes “good”
- Lemmatizers use dictionary lookups and morphological analysis to determine the correct root word. This involves understanding the part of speech and the context around a word, making it more accurate than stemming.
- **Benefits:**
 - Improves accuracy in text processing by grouping related word forms under one lemma.
 - Reduces redundancy to help NLP models better interpret and learn from text data.
 - Helps in better information retrieval and search relevance by matching words with the same meaning.
- **Applications:**
 - Used in NLP pipelines for tasks like text classification, information retrieval, and machine learning preprocessing.
 - Enhances chatbots and AI assistant understanding by normalizing user inputs.
- **Differences from Stemming:**
 - Lemmatization produces valid dictionary words, while stemming may generate non-words.
 - Lemmatization analyzes meaning and context, stemming does not.
 - Lemmatization is more computationally intensive but more precise.

For example:

"Original Sentence: The children are running towards a better place."

"Lemmatized Sentence: The child are run toward a good place."

Lemmatized form based on part of speech:

"children" → "child" (noun)

"running" → "run" (verb)

"better" → "good" (adjective)

"are" remains "are" (auxiliary verb)

This shows that words are normalized to meaningful root forms rather than just stripped endings.

Lemmatization takes into account the grammatical role and meaning rather than just suffix removal.

Penn Treebank

- **Penn Treebank (PTB)** is one of the most important annotated corpora in Natural Language Processing.
- Created by the **Linguistic Data Consortium (LDC)** at the University of Pennsylvania, it provides large, manually annotated datasets used for training and evaluating NLP models.
- It is highly likely to be asked in exams because of its influence on POS tagging and parsing.
- **The Penn Treebank is a large corpus of American English text annotated with syntactic structures, including part-of-speech (POS) tags and phrase-structure parse trees.**
- The Penn Treebank contains:
 - **4.5 million words** of POS-tagged text
 - **Over 2 million words** of parsed text
 - Sources include:
 - The Wall Street Journal (major portion)
 - Brown Corpus samples
 - ATIS (Air Travel Information System) data
 - One of its biggest contributions is the **Penn Treebank POS Tagset**, consisting of **36 tags**.

Examples:

- **NN** – singular noun
- **NNS** – plural noun
- **VB** – base verb
- **VBD** – past tense
- **JJ** – adjective
- **RB** – adverb
- These tags have become industry standards and are used in NLTK, spaCy, Stanford NLP, etc.
- The corpus includes **phrase-structure trees** based on **constituency grammar**, using bracketed notation.

Example:

(S

(NP (DT The) (NN cat))

(VP (VBD sat)

(PP (IN on)

(NP (DT the) (NN mat))))))

The Penn Treebank is crucial because:

- It serves as a **benchmark dataset** for NLP models.
- It standardized **POS tagging** and **syntactic parsing**.
- Many early statistical models (HMM taggers, PCFG parsers) were trained on it.
- Deep-learning based parsers still evaluate on PTB test sets.

Applications

- Training POS taggers
- Training constituency parsers
- Language modeling
- Grammar induction
- Evaluation of NLP algorithms

Brill's Tagger

- Brill's Tagger, developed by **Eric Brill (1995)**, is a rule-based Part-of-Speech (POS) tagging system that uses Transformation-Based Learning (TBL).
- It starts with a simple baseline tagging and then repeatedly applies learned correction rules to improve accuracy.

Key Features

1. **Rule-based but automatically learned** – not manually written.
2. **Transformation-Based Learning (TBL)** – learns rules by analyzing errors.
3. **Starts with a baseline tagger**
 - Common baseline: “Most frequent tag for each word”.
4. **Improves accuracy through transformations** such as:
 - Change NN to VB if the previous word is ‘to’.
5. **Rules are human-readable** and linguistically meaningful.
6. Achieves high accuracy (around 95%) while being simple and interpretable.

How Brill's Tagger Works

1. **Initial tagging:** Assigns a simple POS tag, often based on word frequency.
2. **Error detection:** Compares the tagged text with the correct annotated data.
3. **Rule creation:** Learns correction rules that fix the most errors.
4. **Rule ordering:** The best rules are applied sequentially.
5. **Final tagging:** A refined POS-tagged output after applying all transformations.

Example Transformation Rule

A typical rule looks like:

“Change tag NN to VB if the previous tag is *to*.”

This rule corrects cases like:

- “to **run**”
- “to **swim**”

Where **run** and **swim** should be tagged as verbs (VB), not nouns (NN).

Advantages

- Easy to understand and interpret.
- High accuracy for classical NLP tasks.

- Does not require complex statistical or neural models.
- Rules capture real linguistic patterns.

Limitations

- Slower training because it learns rules iteratively.
- Depends on a good annotated corpus for training.
- Modern deep learning models outperform it in large-scale tasks.

For example:

Initial Baseline Tagging:

to / TO
book / NN
a / DT
ticket / NN

Here, “book” is tagged as **NN (noun)** by the baseline tagger.

But in this sentence, “book” is used as a verb.

So Brill’s Tagger learns a rule:

Rule: *Change NN → VB if the previous word is tagged TO*

After Applying Rule:

to / TO
book / VB
a / DT
ticket / NN

WordNet

- **WordNet** is a large lexical database of English developed at **Princeton University**.
- It groups English words into sets of synonyms called **synsets**, and links these synsets through different **semantic relations**.

Key Concepts

1. Synset

A *synset* is a set of synonymous words expressing a single concept.

Example:

- {car, automobile}

2. Semantic Relations

WordNet defines several important semantic relationships:

- **Hypernymy (superordinate)** → “is-a” relation
 - dog → animal
- **Hyponymy (subordinate)** → “type of”
 - rose → flower
- **Meronymy (part-of)**
 - wheel → part of → car
- **Antonymy**
 - hot ↔ cold
- **Holonymy (whole-of)**
 - tree is holonym of branch

3. Word Senses

A word can have multiple meanings (polysemy). WordNet stores each sense separately.

Example: *bank*

- bank (financial institution)
- bank (river edge)

Structure of WordNet

WordNet is divided into four databases:

1. **Nouns**
2. **Verbs**
3. **Adjectives**
4. **Adverbs**

Each category has its own network of synsets and semantic relations.

Importance of WordNet

- Provides semantic knowledge needed for many NLP tasks
- Useful for word sense disambiguation (WSD)
- Helps in semantic similarity measurement
- Supports information retrieval, chatbots, text classification, and ontology building
- Available in popular NLP libraries like NLTK, spaCy, and WordNet API

Example of WordNet

For the word **car**:

- Synset: {car, auto, automobile, machine, motorcar}
- Hypernym: vehicle
- Hyponym: taxi, police car, race car
- Meronym: engine, wheel, door

It shows how WordNet organizes word meanings in a conceptual network.

PropBank

- PropBank (Proposition Bank) is an annotated corpus that provides semantic role labeling (SRL) for verbs.
- Its primary purpose is to add semantic role annotations to verb instances, which support various NLP tasks such as semantic parsing, information extraction, and question answering
- It adds a layer of semantic information on top of syntactic structures (often from the Penn Treebank) by identifying who did what to whom in a sentence.

Key Concepts

1. Predicate and Arguments

- **Predicate:** Typically a verb that represents an action or event.
- **Arguments:** Entities participating in the action, labeled consistently across sentences.

Example for verb *give*:

- Arg0 → giver (agent)
- Arg1 → thing given (theme)
- Arg2 → receiver (recipient)

2. Frames

Each verb has a **frame** describing its possible argument structures and optional roles.

- Example: *sell* → {seller, item, buyer}

3. Annotated Corpus

- PropBank annotates verbs in sentences with **semantic roles**, not just POS tags.
- Often built on top of the **Penn Treebank**, aligning semantic roles with syntactic structures.

For example:

Word	POS	PropBank Role
John	NNP	Arg0 (Seller)
sold	VBD	Predicate
a car	NN	Arg1 (Thing sold)
to Mary	NNP	Arg2 (Buyer)

FrameNet

- FrameNet is a lexical database of English based on frame semantics, developed at the International Computer Science Institute (ICSI).
- It organizes words according to the conceptual frames they evoke, describing situations, events, or states along with participants and properties.
- Captures deep semantic meaning beyond syntactic roles.
- Supports semantic parsing, information extraction, and NLP applications requiring understanding of events.
- Complements resources like WordNet and PropBank by providing richer context for words.
- Helps in building knowledge graphs, chatbots, and AI understanding systems.

Key concepts

1. Frames

A **frame** represents a **conceptual scenario** or event and defines **frame elements (roles)**.

- Example: *Commerce_buy* frame
 - Buyer, Seller, Goods, Money

2. Lexical Units

Words or phrases that evoke a frame are called **lexical units**.

- Example: *buy, purchase, acquire* → evoke the *Commerce_buy* frame

3. Frame Elements

Frame elements are the **participants or attributes** in the frame:

- Mandatory: core participants (Buyer, Seller, Goods)
- Optional: adjuncts like Time, Place, Manner

4. Annotated Corpus

FrameNet provides **manually annotated sentences** showing how frame elements are realized in actual text.

For Example

Sentence:

“Mary bought a book from John yesterday.”

Identify the Frame

- **Frame:** *Commerce_buy* (represents the event of buying/selling)

- **Lexical Unit (Predicate):** bought

Identify Frame Elements

Frame Element	Word/Phrase in Sentence	Role Description
Buyer	Mary	Person performing the purchase
Goods	a book	Item being bought
Seller	John	Person selling the item
Time	yesterday	When the event occurred (adjunct)

- FrameNet links words (lexical units) to conceptual frames.
- Frame elements are participants or attributes in the frame.
- Core elements (Buyer, Goods, Seller) are mandatory for the event; adjuncts (Time) are optional.

Representation

Commerce buy

Buyer: Mary

Seller: John

Goods: a book

Time: yesterday

Predicate: bought

- It focuses on the conceptual meaning of the event rather than just the syntactic roles.

Brown Corpus

- The Brown Corpus is one of the first large, balanced corpora of American English, developed at Brown University in the 1960s.
- It contains about 1 million words from various genres and is manually annotated with part-of-speech tags. It was designed to provide a representative sample of written American English.

Key Features

1. **Size:** ~1 million words
2. **Genres Covered:**
 - Press reportage (news)
 - Fiction
 - Religion
 - Academic writing
 - Miscellaneous
3. **POS Tagging:** Words are annotated with part-of-speech tags.
4. **Balanced Corpus:** Carefully sampled to represent different styles and topics of English.
5. **Manual Annotation:** High-quality human tagging ensures reliability.

Importance of Brown Corpus

- First standardized corpus in computational linguistics.
- Used for statistical analysis of English, e.g., word frequency, collocations, and grammar patterns.
- Provides training data for POS taggers, parsers, and language models.
- Inspired later corpora such as Penn Treebank, BNC, and COCA.
- Useful for studying language standardized corpus variation across genres.

Example

Sentence 1 (from fiction):

“The quick brown fox jumps over the lazy dog.”

POS Tags:

The/DT quick/JJ brown/JJ fox/NN jumps/VBZ over/IN the/DT lazy/JJ dog/NN

Sentence 2 (from news genre)

“The president addressed the nation today.”

POS Tags:

The/DT president/NN addressed/VBD the/DT nation/NN today/NN

Applications in NLP:

- Training POS taggers
- Studying word frequencies
- Statistical language modeling

British National Corpus (BNC)

- The British National Corpus (BNC) is a 100-million-word collection of samples of spoken and written British English.
- Compiled in the 1990s, it aims to represent modern British English across a wide range of genres and contexts.
- Provides a large-scale, balanced resource for corpus linguistics.
- Used for frequency analysis, collocation studies, concordances, and word sense analysis.
- Supports NLP applications like POS tagging, parsing, text classification, and language modeling.
- Serves as a benchmark for modern British English.
- Complements other corpora like the Brown Corpus (American English) and Penn Treebank (POS & parsing annotated corpus).

Key Features

1. **Size:** 100 million words
2. **Language:** British English
3. **Spoken and Written Texts:**
 - Spoken: conversations, interviews, radio broadcasts
 - Written: newspapers, fiction, academic writing, organizational documents
4. **Balanced Corpus:** Includes a representative mix of genres, topics, and styles.
5. **Annotation:**
 - Part-of-speech tagging
 - Lemmatization
 - Metadata on speaker demographics, text source, and context
6. **Purpose:** To study patterns of contemporary English usage systematically.

Example of BNC Annotation:

Sentence (spoken or written):

“The conference will be held next week in London.”

POS Tags:

The/DT conference/NN will/MD be/VB held/VBN next/JJ week/NN in/IN London/NNP

Metadata Example:

- Genre: Written, Academic
 - Text Source: Conference announcement
 - Speaker Demographics (for spoken part): Age, Gender, Region
-
- BNC includes linguistic annotation (POS, lemma) and contextual metadata, which makes it richer than Brown Corpus for linguistic analysis and NLP tasks.